



Première année Science du Numérique : Technologie Objet

Rapport de Projet : NutriCoach

Assistant de Régulation Métabolique
par l'équilibre Nutrition / Sport / Plaisir

Équipe 2 :

M. Mickael FAURE
M. Arthur GELINEAU
M. Louis LEBRETON
M^{me} Jeanne MONTMERLE
M. Arthur MORÉ
M. Lucas MOUSSY
M. Alexandre PONCIN
M. Justin ROUX

Encadrants :

M. Xavier CREGUT
M. Olivier DUFFORT

Version du
23 mai 2026

Table des matières

1	Introduction et Rappel du Sujet	2
2	Organisation de l'Équipe et Méthodes Agiles	2
2.1	Répartition des rôles	2
2.2	Méthode de travail	2
3	Architecture Globale et Découpage en Paquetages	3
3.1	Architecture globale	3
3.2	Diagrammes UML de classes	3
4	Fonctionnalités et Implémentation (Vue par Module)	5
4.1	Module NUTRITION	5
4.1.1	Fonctionnalités	5
4.1.2	Architecture des classes	5
4.2	Module STATS	5
4.2.1	Fonctionnalités	5
4.2.2	Modèle mathématique : la courbe logarithmique	5
4.2.3	Algorithme de lissage	6
4.3	Module PLANNING	6
4.3.1	Fonctionnalités	6
4.3.2	Architecture des classes	6
4.3.3	Algorithme de génération et de régulation	6
4.4	Module EXTRA	7
4.4.1	Extra nutritionnel (Cheat Meal)	7
4.4.2	Extra sportif	8
4.4.3	Architecture du SmartPlanner	9
4.5	Module COMPTE	9
4.5.1	Fonctionnalités	9
4.5.2	Persistence des données	9
4.6	Module BLIND MODE	9
4.6.1	Motivation	9
4.6.2	Architecture : Pattern Strategy + Observer	10
5	Dynamique du Système	10
5.1	Diagramme de séquence : validation d'une séance	10
5.2	Diagramme de séquence : activation du Blind Mode	10
6	Choix de Conception, Problèmes Rencontrés et Solutions	11
6.1	Architecture : la façade FctAppModules	11
6.2	Persistence : séparation profil statique / données dynamiques	11
6.3	Modèle de projection du poids	11
6.4	Le « Bouclier Temporel » dans le planning	11
6.5	Système de Streak	12
6.6	Taux de compensation du SmartPlanner	12
6.7	Restructuration de l'arborescence et correction des imports	12
6.8	Choix de la formule calorique	12

1 Introduction et Rappel du Sujet

NutriCoach est un **assistant de régulation métabolique** développé en Java (interface Swing), dont l'objectif est d'aider l'utilisateur à atteindre ses objectifs physiques — remise en forme, perte de poids ou prise de masse — de manière saine, durable et déculpabilisante.

Contrairement aux applications de comptage de calories classiques qui imposent des restrictions rigides, NutriCoach fait le lien entre l'apport nutritionnel et la dépense sportive en s'appuyant davantage sur l'activité physique que sur la privation. Si l'utilisateur s'écarte de son plan alimentaire, l'application ne le sanctionne pas : elle recalcule dynamiquement le programme sportif (en ajustant la durée ou l'intensité d'une séance) pour lisser l'écart. L'objectif final est de préserver la santé mentale — prévention des Troubles du Comportement Alimentaire (TCA) — et physique de l'utilisateur tout en garantissant la convergence vers son objectif initial.

L'application fonctionne entièrement en local à partir de fichiers CSV (base nutritionnelle et sportive) et de fichiers texte de comptes.

2 Organisation de l'Équipe et Méthodes Agiles

2.1 Répartition des rôles

Le projet a été développé en trois itérations successives. Le tableau suivant résume la répartition principale des responsabilités.

Membre	Rôle principal
Justin Roux	Interface graphique Swing complète, couche de liaison UI/modules (<code>FctAppModules</code>), restructuration du dépôt
Lucas Moussy	Module <code>evolutionStatistique</code> , modèle de projection logarithmique, intégration graphique
Arthur Gelineau	Module <code>gestionPlanning</code> , algorithme <code>ManagerPlanning</code> , persistance des plannings
Arthur Moré	Module <code>assistantNutritionnel</code> , base de données des recettes, interface <code>AssistantAdmin</code>
Louis Lebreton	Module <code>blindMode</code> — <code>DisplayManager</code> , <code>DisplayStrategy</code> , tests JUnit 4
Jeanne Montmerle	Module <code>blindMode</code> — <code>InputAdapter</code> , <code>BlindInputAdapter</code> , <code>UserProfile</code> , <code>BlindStrategy</code>
Alexandre Poncin	Connexion UI/assistant nutritionnel, enrichissement de la base de données CSV
Mickael Faure	Module <code>smartPlanner</code> , module <code>calculCalories</code> , intégration et tests

2.2 Méthode de travail

L'équipe a adopté une organisation proche du framework **Scrum** avec trois sprints de durée fixe. Un **Google Sheets partagé** a servi de backlog : chaque tâche y était décrite avec son responsable, sa localisation dans le code, son état d'avancement et un coefficient de priorité.

Les itérations se sont organisées ainsi :

Itération 1 Conception UML, prototypage CLI de chaque module métier, premières ébauches de l'interface Swing.

Itération 2 Développement complet de l'interface Swing (J. Roux), premières intégrations UI/modules, tests JUnit du Blind Mode.

Itération 3 Intégration finale de tous les modules, persistance robuste des données, corrections architecturales et livraison.

3 Architecture Globale et Découpage en Paquetages

3.1 Architecture globale

Le projet suit une architecture en **quatre couches** :

Paquetage	Rôle
main/	Point d'entrée (Main.java), initialisation des dossiers
ui/	Interface graphique Swing : pages, popups, navigation, thèmes
modules/	Logique métier répartie en six sous-modules indépendants
fonctions/	Couche de liaison UI ↔ modules (FctAppModules) et I/O fichiers (GestionFichier)

La façade **FctAppModules** est le **seul point de contact** entre l'interface graphique et les modules métier. Aucune page Swing ne dépend directement d'un module : cela garantit que brancher un nouveau module en itération 3 n'imposait de modifier que **FctAppModules**, sans toucher aux pages existantes.

3.2 Diagrammes UML de classes

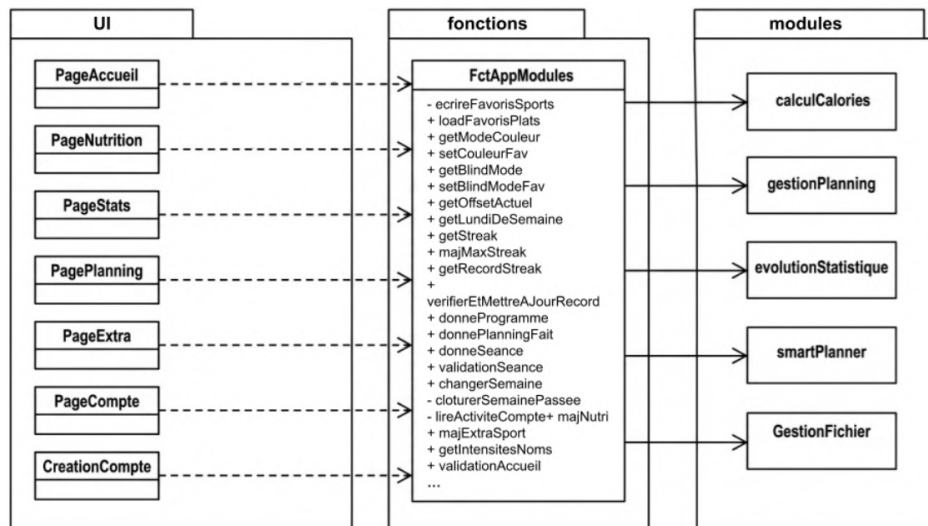


FIGURE 1 – Vue d'ensemble du projet.

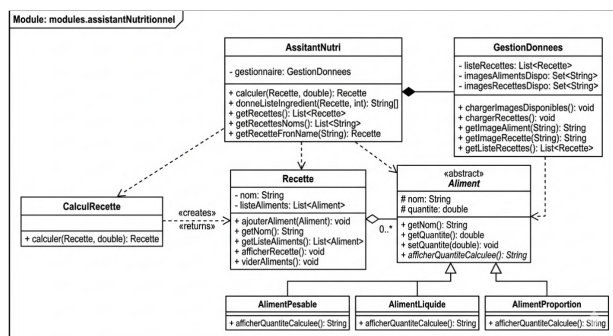


FIGURE 2 – Module assistantNutritionnel.

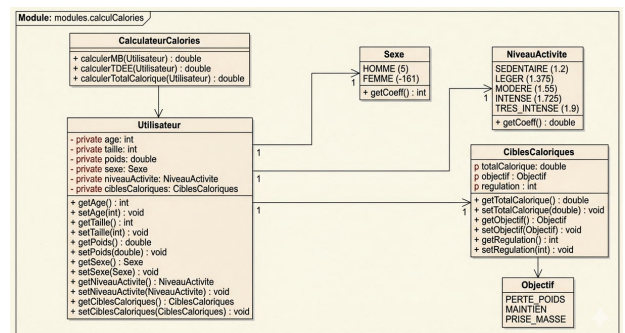
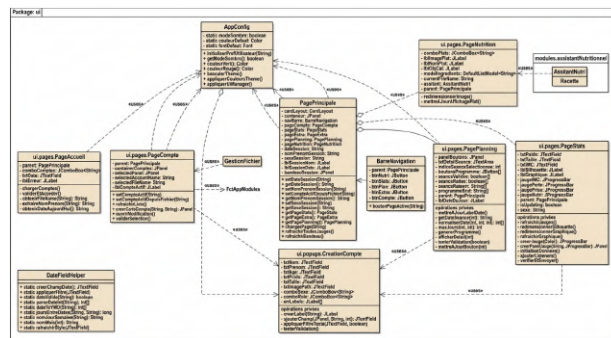
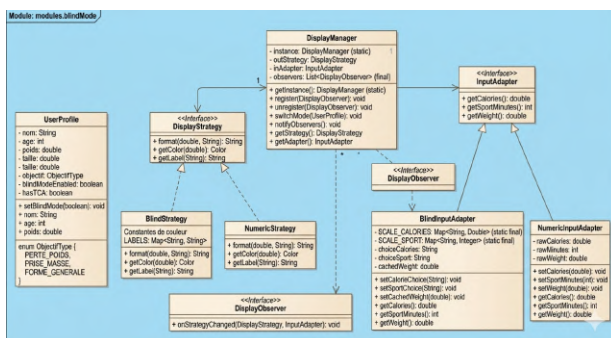
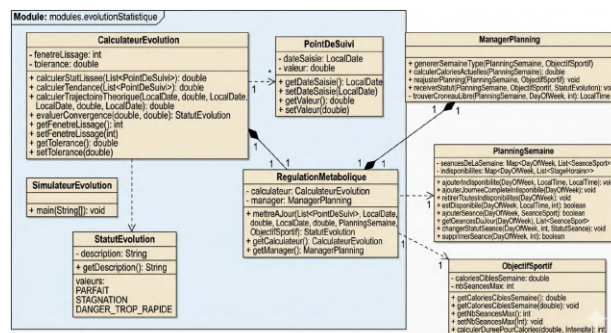
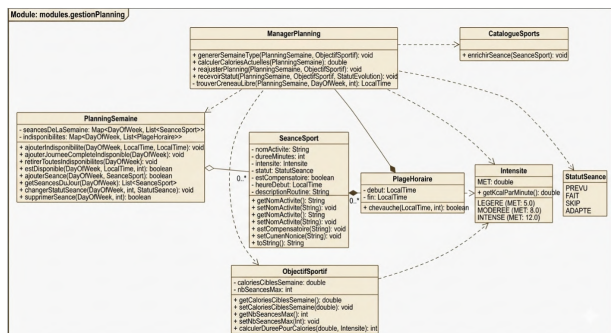


FIGURE 3 – Module calculCalories.



4 Fonctionnalités et Implémentation (Vue par Module)

Dans cette section, nous détaillons l'architecture et l'état d'avancement des fonctionnalités en suivant la structure de navigation principale de l'application.

4.1 Module NUTRITION

4.1.1 Fonctionnalités

- Recherche de recettes en temps réel par nom (filtrage dynamique).
- Ajustement automatique des portions selon l'objectif calorique de l'utilisateur.
- Système de recettes favorites avec persistance.
- Copie de la liste d'ingrédients adaptée à l'objectif.
- Affichage de l'image du plat sélectionné.

4.1.2 Architecture des classes

Le module repose sur une hiérarchie polymorphique pour représenter les aliments selon leur unité de mesure :

- **Aliment** (classe abstraite) — porte le nom et la quantité de base pour 100 kcal ; impose la méthode `afficherQuantiteCalculee()`.
- **AlimentPesable**, **AlimentLiquide**, **AlimentProportion** — trois spécialisations concrètes avec leur propre formatage (grammes, millilitres, portions).
- **Recette** — agrège une liste d'**Aliment**.
- **CalculRecette** — ajuste dynamiquement les quantités d'une recette pour atteindre un objectif calorique cible.
- **GestionDonnees** — charge les recettes depuis `plats.csv` et scanne les dossiers d'images.
- **AssitantNutri** — façade du module, utilisée directement par **PageNutrition**.

La formule de recalcul est simple : pour un objectif de C kcal et une recette de base à C_0 kcal, chaque ingrédient voit sa quantité multipliée par C/C_0 .

4.2 Module STATS

4.2.1 Fonctionnalités

- Saisie du poids et de la taille avec calcul de l'IMC en temps réel.
- Graphique d'évolution dynamique (courbe réelle + courbe prédictive théorique superposées).
- Projection du poids futur à une date donnée.
- Affichage d'une silhouette adaptée au sexe de l'utilisateur.

4.2.2 Modèle mathématique : la courbe logarithmique

En itération 1, le modèle de trajectoire théorique était linéaire ($y = at + b$). Ce modèle lissait bien les objectifs psychologiquement mais créait une asymptote irréaliste sur le long terme : la perte de poids ralentit biologiquement.

En itération 3, ce modèle a été remplacé par une **courbe logarithmique continue** (`CalculateurEvoluti`

$$poids(t) = p_0 + (p_c - p_0) \times \frac{\ln(t + 1)}{\ln(T + 1)}$$

où t est le nombre de jours écoulés, T la durée totale du programme, p_0 le poids initial et p_c le poids cible. Le terme $+1$ évite le $\ln(0)$. Ce modèle reflète les **rendements décroissants** du métabolisme : la progression est plus rapide en début de programme, puis se stabilise, sans jamais totalement stagner.

4.2.3 Algorithme de lissage

Pour éliminer le bruit des fluctuations quotidiennes normales (rétention d'eau, pesées de matin/soir), `CalculateurEvolution` applique une **moyenne glissante** sur une fenêtre de N jours (`calculerStatLisee`), complétée d'une **régression linéaire par moindres carrés** (`calculerTendance`) pour extraire la pente réelle d'évolution.

4.3 Module PLANNING

4.3.1 Fonctionnalités

- Génération automatique du programme hebdomadaire à partir de l'objectif calorique et du niveau d'activité.
- Validation / échec de séances avec recalcul automatique du planning restant.
- Navigation temporelle (semaines passées en lecture seule, séances non validées clôturées automatiquement).
- Système de **Streak** : compteur de séances consécutives validées avec record absolu.
- Prise en compte des sports favoris pour la génération du planning.
- Adaptation du volume au profil (sédentaire, modéré, athlétique).

4.3.2 Architecture des classes

- `PlanningSemaine` — structure centrale : `Map<DayOfWeek, List<SeanceSport>` et `Map<DayOfWeek, List<PlageHoraire>` pour les indisponibilités.
- `SeanceSport` — modélise une séance (activité, durée, intensité, statut, heure, flag compensatoire).
- `Intensite` (enum) — FAIBLE / MODERE / INTENSE, chacun avec son coefficient kcal/min.
- `StatutSeance` (enum) — PLANIFIEE / VALIDEE / RATEE / COMPENSATOIRE.
- `ManagerPlanning` — le « solveur » : génère la semaine type et réajuste dynamiquement en cas de déficit calorique.
- `GestionPlanningDonnees` — persistance I/O des plannings (fichiers texte nommés `Prenom_NOM_planni`).

4.3.3 Algorithme de génération et de régulation

`ManagerPlanning.genererSemaineType()` répartit l'objectif calorique hebdomadaire en séances sous contraintes : espacement minimal pour la récupération musculaire, durée plafonnée à 90 minutes, et respect des plages d'indisponibilité. En cas de séance ratée, l'algorithme applique trois stratégies de compensation par ordre de priorité :

1. Décalage de la séance vers un jour de repos disponible.
2. Augmentation de l'intensité des séances restantes de la semaine.
3. Rallongement de la durée des séances existantes.

Un **bouclier temporel** garantit que l'algorithme ne modifie jamais le passé : le paramètre `jourActuel` est transmis à `reajusterPlanning()` pour restreindre strictement les modifications aux jours futurs.

4.4 Module EXTRA

La page Extra permet à l'utilisateur de renseigner des informations énergétiques ponctuelles hors programme.

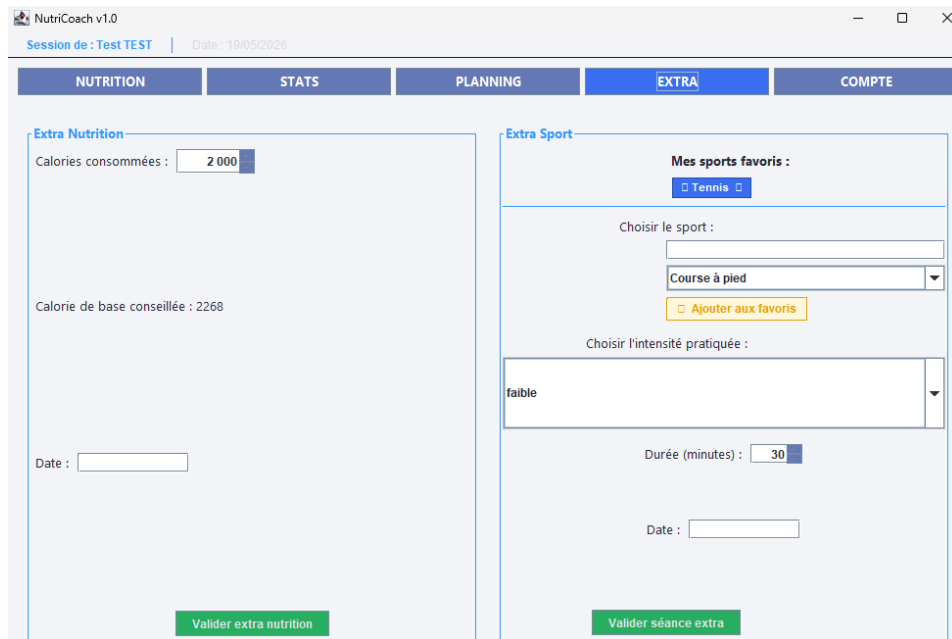


FIGURE 8 – Page Extra.

La page est divisée en deux parties : à gauche, l'ajout d'un apport nutritionnel additionnel (calories), et à droite, le renseignement d'une séance de sport réalisée indépendamment du planning.

4.4.1 Extra nutritionnel (Cheat Meal)

Lorsque l'utilisateur renseigne un excès calorique, le **SmartPlanner** est invoqué. Il propose au maximum 3 options de compensation sportive dans une boîte de dialogue :

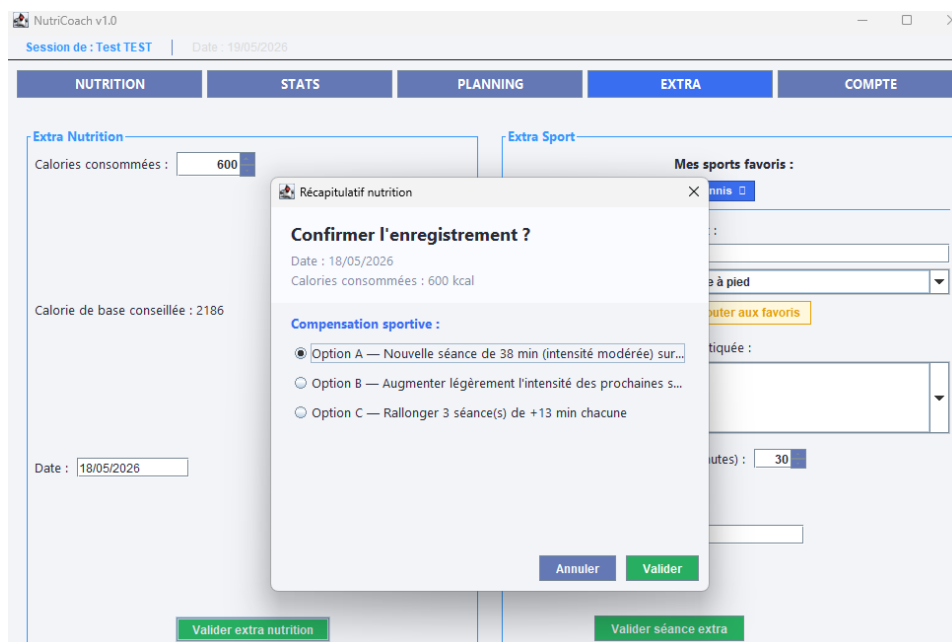


FIGURE 9 – Validation d'un extra nutritionnel.

— Ajout d'une nouvelle séance de sport sur un créneau libre.

- Augmentation de l'intensité de séances existantes.
- Rallongement de la durée de séances existantes.

Le **taux de compensation est fixé à 50 %** : un extra de 600 kcal ne génère une compensation que de ≈ 300 kcal par le sport. Ce choix délibéré évite que la compensation soit perçue comme une punition et préserve l'équilibre psychologique de l'utilisateur.

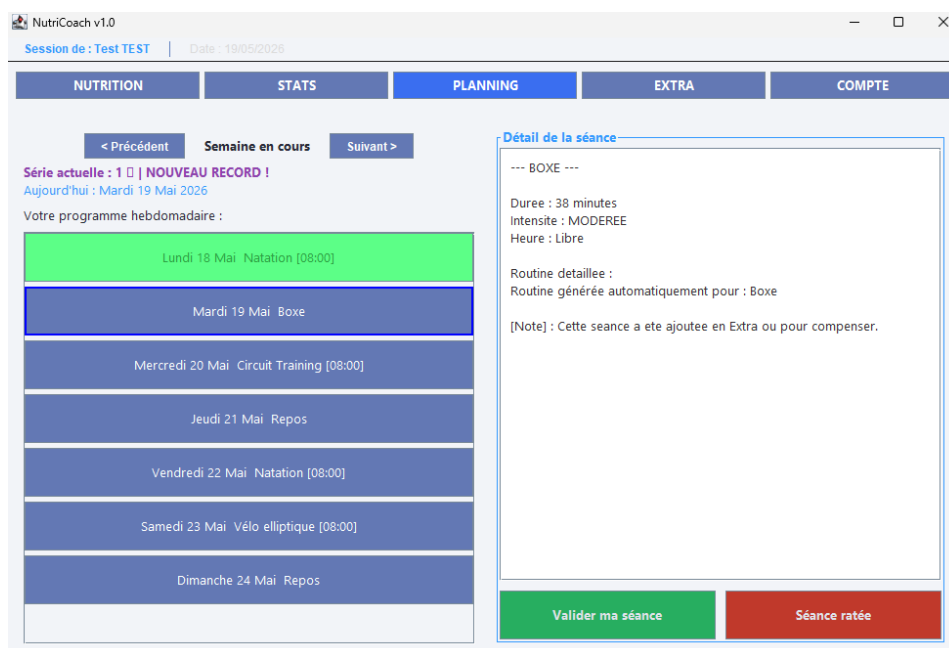


FIGURE 10 – Séance de compensation ajoutée au planning.

4.4.2 Extra sportif

Lorsqu'une séance de sport est réalisée hors planning, l'utilisateur la saisit dans la partie droite. Aucune compensation n'est proposée (baisser les calories ou modifier les séances serait perçu comme punitif). La séance est directement ajoutée au planning avec l'annotation [Extra].

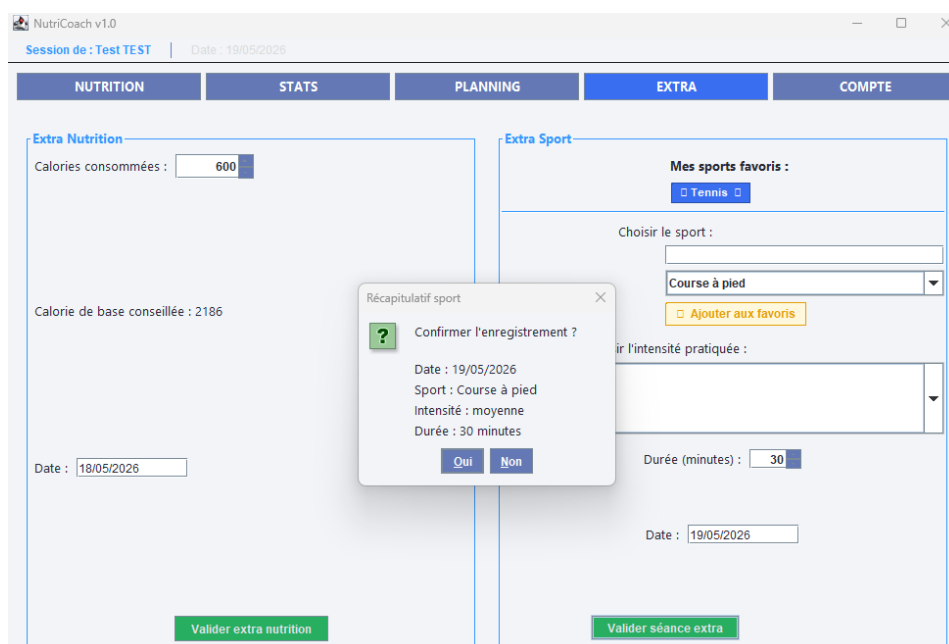


FIGURE 11 – Validation d'un extra sportif.



FIGURE 12 – Séance extra visible dans le planning.

4.4.3 Architecture du SmartPlanner

- `SmartPlanner` — génère les scénarios et les applique au `PlanningSemaine` via `ManagerPlanning`.
- `Scenario` — encapsule un libellé, une stratégie (`NOUVELLE_SEANCE` / `BOOST_INTENSITE` / `RALLONGEMENT`) et un nombre de calories à compenser.
- `EcartAlimentaire` — représente l'écart saisi (date, calories en surplus, type, description).
- `StatutCompensation` (enum) — `EN_ATTENTE` / `APPLIQUE` / `ANNULE`.

4.5 Module COMPTE

4.5.1 Fonctionnalités

- Création, modification et suppression de comptes utilisateurs.
- Authentification par mot de passe.
- Paramétrage du profil : âge, sexe, poids, taille, photo, niveau d'activité, objectif, sports favoris.
- Bascule du thème clair/sombre (`AppConfig`).
- Activation du Blind Mode.

4.5.2 Persistance des données

Chaque compte est stocké dans un fichier texte nommé selon la convention `Prenom_NOM_U.txt` (U = utilisateur, C = coach, etc.). Les données dynamiques (historique des pesées, objectif, records de streak) sont séparées dans un fichier `_stats.txt` dédié. Ce découpage applique le principe de **séparation des responsabilités** et évite les conflits de fusion sur le dépôt Git lors du travail en équipe.

4.6 Module BLIND MODE

4.6.1 Motivation

Certains utilisateurs ayant un passif de TCA ressentent de l'anxiété face aux chiffres (calories exactes, grammes, pourcentages). Le Blind Mode remplace les valeurs numériques par des indicateurs qualitatifs (codes couleur, jauges, libellés comme « Énergie optimale »).

4.6.2 Architecture : Pattern Strategy + Observer

Le module s'appuie sur deux design patterns combinés :

Strategy (affichage) :

- `DisplayStrategy` (interface) — définit `format(double, String)`, `getColor(double)`, `getLabel(String)`.
- `NumericStrategy` — retourne les valeurs numériques brutes (mode normal).
- `BlindStrategy` — retourne des libellés qualitatifs et des couleurs sans chiffres.

Adapter (saisie) :

- `InputAdapter` (interface) — `getCalories()`, `getSportMinutes()`, `getWeight()`.
- `NumericInputAdapter` — saisie directe de valeurs numériques.
- `BlindInputAdapter` — convertit des choix qualitatifs (« repas léger ») en valeurs numériques exploitables.

Observer (propagation) :

- `DisplayManager` (singleton) — maintient la stratégie et l'adaptateur actifs; notifie tous les `DisplayObserver` lors d'un changement de mode.
- `PageStats`, `PageNutrition`, `PagePlanning`, `PageExtra` implémentent `DisplayObserver` et se mettent à jour automatiquement via `onStrategyChanged()`.

Cette architecture garantit qu'un changement de mode se propage instantanément à toute l'interface sans couplage direct entre les pages.

5 Dynamique du Système

5.1 Diagramme de séquence : validation d'une séance

Le scénario suivant décrit le flux complet déclenché lorsque l'utilisateur clique sur « Valider » une séance dans `PagePlanning` :

1. `PagePlanning.tenterValidation(true)` est appelée.
2. Elle appelle `FctAppModules.majExtraSport()` ou `majStatutSeance()` selon le type de séance.
3. `FctAppModules` met à jour le statut de la `SeanceSport` dans le `PlanningSemaine` en mémoire.
4. `ManagerPlanning.reajusterPlanning()` est invoqué pour compenser d'éventuels déficits.
5. `GestionPlanningDonnees.sauvegarderPlanning()` persiste l'état sur disque.
6. `FctAppModules` recalcule le streak via `getStreak()`.
7. `PagePlanning.genererProgramme()` recharge les données fraîches depuis la façade et redessine tous les boutons de la semaine.

5.2 Diagramme de séquence : activation du Blind Mode

1. L'utilisateur active le toggle dans `PageCompte`.
2. `FctAppModules.setBlindModeFav(user, true)` persiste la préférence.
3. `DisplayManager.switchMode(userProfile)` est appelé.
4. `DisplayManager` remplace `NumericStrategy` par `BlindStrategy` et `NumericInputAdapter` par `BlindInputAdapter`.
5. `DisplayManager.notifyObservers()` déclenche `onStrategyChanged()` sur toutes les pages enregistrées.
6. Chaque page rafraîchit son affichage sans aucun rechargement global.

6 Choix de Conception, Problèmes Rencontrés et Solutions

6.1 Architecture : la façade FctAppModules

Choix : Dès l'itération 2, FctAppModules a été conçue comme unique point de contact entre l'UI et les modules. Ce choix de **pattern Façade** a permis à chaque développeur de module de travailler de manière totalement indépendante, sans se soucier de l'interface graphique.

6.2 Persistance : séparation profil statique / données dynamiques

Problème rencontré

Les conflits Git étaient fréquents lorsque plusieurs personnes modifiaient simultanément le fichier de compte d'un utilisateur, qui contenait à la fois des données de profil fixes et un historique évoluant à chaque session.

Solution apportée

Création d'un fichier `_stats.txt` dédié à l'historique des pesées, à l'objectif en cours et aux records de streak, séparé du fichier de profil. Cette architecture respecte le principe de séparation des responsabilités (SOLID) et supprime les conflits de fusion.

6.3 Modèle de projection du poids

Problème rencontré

Le modèle linéaire initial ($y = at + b$) créait une asymptote irréaliste : la projection indiquait une perte de poids continue et constante même sur 10 ans, ce qui ne correspond pas à la réalité biologique.

Solution apportée

Remplacement par un modèle logarithmique ($poids(t) = p_0 + (p_c - p_0) \cdot \frac{\ln(t+1)}{\ln(T+1)}$), qui modélise les rendements décroissants du métabolisme tout en restant mathématiquement continu.

6.4 Le « Bouclier Temporel » dans le planning

Problème rencontré

L'algorithme de régulation de ManagerPlanning pouvait, après correction d'une séance, placer des séances de rattrapage dans le passé (jours déjà écoulés de la semaine), rendant le planning incohérent.

Solution apportée

Ajout du paramètre `jourActuel` (DayOfWeek) dans `reajusterPlanning()`. Une condition stricte force l'algorithme à ne modifier que les jours futurs de la semaine en cours.

6.5 Système de Streak

Problème rencontré

L'algorithme initial de calcul du streak additionnait naïvement toutes les séances au statut **VALIDEE** sans tenir compte des séries brisées : une semaine avec 3 séances validées, 1 ratée et 2 validées affichait un streak de 5 au lieu de 2.

Solution apportée

Refonte de `getStreak()` pour un parcours strictement chronologique des séances. Le compteur retombe à 0 dès qu'un statut **SKIP** est rencontré. Ajout d'un record absolu sauvegardé dynamiquement et affiché sur l'interface.

6.6 Taux de compensation du SmartPlanner

Problème rencontré

La première version du SmartPlanner compensait intégralement chaque écart calorique par le sport. Un extra de 600 kcal génèrait des séances de compensation disproportionnées, perçues comme une punition par l'utilisateur.

Solution apportée

Introduction d'un taux de compensation fixé à 50 % (`TAUX_COMPENSATION = 0.5`). Un extra de 600 kcal génère une compensation d'environ 300 kcal par le sport. Ce choix est cohérent avec la philosophie anti-culpabilisation du projet.

6.7 Restructuration de l'arborescence et correction des imports

Problème rencontré

En début d'itération 3, les fichiers sources n'étaient pas organisés en paquetages cohérents. La restructuration complète de l'arborescence a cassé l'ensemble des déclarations `package` et des `import` sur plus d'une cinquantaine de fichiers simultanément.

Solution apportée

Correction en masse des imports et des chemins d'accès aux ressources (`data/`), avec un recours ciblé à l'IA pour les tâches purement répétitives de remplacement. La logique de conception et l'architecture sont restées entièrement gérées par l'équipe.

6.8 Choix de la formule calorique

Le module `calculCalories` utilise la **formule de Mifflin St-Jeor** pour calculer le métabolisme de base (MB), retenue après comparaison avec les formules de Harris-Benedict et Katch-McArdle pour sa précision sur des populations mixtes :

$$MB = 10 \times poids + 6,25 \times taille - 5 \times ge + coeff_sexe$$

Le **TDEE** (Total Daily Energy Expenditure) est obtenu en multipliant le MB par le coefficient du niveau d'activité (1,2 pour sédentaire à 1,9 pour très intense). Le total calorique cible ajoute ou soustrait une régulation selon l'objectif (perte de poids, maintien, prise de masse).